

JAYS SHOP

Digital Retail Platform — Engineering Report

HERO-MEDIA UPLOAD INCIDENT & INFRASTRUCTURE COST REVIEW

Root-cause analysis of the R2 upload failures, the diagnostic path taken, the fix that resolved it, and a factual review of current Vercel / Cloudflare R2 / Neon subscription usage and limits.

Prepared for the Project Owner — July 27, 2026

Part 1 — Incident Report: Hero Media Upload Failures

1. Executive Summary

On July 27, 2026, admin hero video uploads were failing in the browser with a CORS error, and hero image uploads were failing with an HTTP 500. These looked like two unrelated bugs but shared a single root cause: a stray whitespace/newline character had been pasted into the **R2_ACCESS_KEY_ID** or **R2_SECRET_ACCESS_KEY** environment variable in Vercel, most likely a copy-paste artifact from the Cloudflare dashboard. The fix (commit **06b2db2**) trims all five R2 environment variables before they are used, and the issue is resolved — both image and video hero-media uploads are confirmed working.

2. Symptoms as Observed

- Admin hero **video** upload failed in the browser with a console error: *"blocked by CORS policy: No 'Access-Control-Allow-Origin' header"*.
- Admin hero **image** upload returned an HTTP **500 Internal Server Error**.
- The two failures presented through entirely different code paths and looked like separate bugs. They were not — both traced back to the same corrupted credential.

3. Root Cause

A stray whitespace or newline character was present inside the **R2_ACCESS_KEY_ID** and/or **R2_SECRET_ACCESS_KEY** values stored in Vercel's environment variable settings. This class of defect is a common copy-paste artifact: Cloudflare's dashboard "Copy" button, or an intermediate paste through an editor/terminal, can silently append a trailing newline or space that is invisible in the Vercel dashboard's text input but is preserved byte-for-byte at runtime.

Why one bad character produced two different-looking failures comes down to how the two upload paths authenticate with Cloudflare R2:

- **Image uploads (real SDK call):** the image goes through the Next.js server — multipart parsing → `saveUploadedFile` → `saveToR2` → the AWS SDK's `PutObjectCommand`. This builds a real HTTP request with an `Authorization` header containing the access key. Node.js's `http.setHeader` rejects illegal characters (like an embedded newline) in header values outright, throwing `TypeError [ERR_INVALID_CHAR]`. That uncaught error surfaced to the browser as a 500.
- **Video uploads (presigned URL):** the browser uploads directly to R2 using a presigned URL, where the SigV4 signature is embedded in the URL's **query string**, never in a header. Generating that URL doesn't touch `setHeader` at all, so presigning always succeeded and looked completely healthy. But the signature computed from the corrupted secret key was silently wrong. When the browser then issued the actual PUT to that presigned URL, R2 rejected it as an authentication failure — and R2's error response, like most auth-rejection responses, does not include an `Access-Control-Allow-Origin` header. The browser has no way to distinguish "the server didn't send CORS headers because it's misconfigured" from "the server didn't send CORS headers because it rejected the request for an unrelated reason," so it reported a CORS failure. **The CORS error was never a CORS problem** — it was an authentication failure with a misleading browser-side label.

4. The Diagnostic Journey — Including the Dead Ends

This section is included deliberately: two earlier, genuinely real issues were investigated and fixed along the way, but neither was the actual root cause. Documenting the dead ends is more useful than hiding them, since the same investigative pattern (a real symptom, a real partial fix, and a still-open failure) is likely to recur.

4.1 CORS policy was genuinely missing at first

Early in the investigation, the R2 bucket had no CORS configuration at all, which would by itself block direct browser PUT requests to R2. This was real and was fixed by adding `configureR2Cors()` (commit **52e071c**), applied via a one-off script against the Cloudflare dashboard/API. The fix was later verified live with a direct `curl -X OPTIONS` preflight request against the R2 endpoint, which returned **204** with the correct `Access-Control-Allow-Origin` headers present — confirming the CORS policy itself was correctly applied and was not the ongoing cause of the failures that persisted afterward.

4.2 The empty-body CRC32 checksum bug (real bug, fixed, not the root cause)

The AWS SDK v3 (the project is on `@aws-sdk/client-s3` `^3.1095.0`) defaults `requestChecksumCalculation` to `WHEN_SUPPORTED`. When presigning a `PutObjectCommand` with no `Body` set (correct here, since the browser supplies the real bytes later), the SDK computed a CRC32 checksum of an *empty* payload and baked `x-amz-checksum-crc32=AAAAAA==` into the presigned URL's signature. Since the browser's real upload body was never empty, R2 rejected the mismatched checksum. This was fixed in commit **11d9736** by setting `requestChecksumCalculation: 'WHEN_REQUIRED'` on the S3 client. This was a real, independent bug and the fix is correct and still in place — but fixing it did not resolve the credential-corruption failures, since those were caused by a different mechanism entirely.

4.3 The doubled-bucket-path theory (turned out to be wrong)

Cloudflare's dashboard displays the R2 "S3 API" endpoint with the bucket name already appended as a path suffix (e.g. `https://<account>.r2.cloudflarestorage.com/<bucket>`). Because the SDK is configured with `forcePathStyle: true` (which itself appends `/<bucket>/<key>`), the theory was that pasting the dashboard's URL verbatim into `R2_ENDPOINT` would double the bucket segment in the final request path, producing something like `/jayshop-media/jayshop-media/<key>`, which R2 would reject with a 400 that (again) omits CORS headers and looks like a CORS failure in the browser. Commit **4c086ac** added `normalizeR2Endpoint()` to strip a trailing `/<bucketName>` from the endpoint defensively.

This theory turned out to be wrong.

Server-side logging of the presigned URL's pathname (added in commit `d2e281f`, described below) proved the request path was always correctly single-bucket — there was never a doubled path in production. The normalization fix is harmless and remains in place as a defensive guard against a real future misconfiguration, but it was not what fixed the incident. It is stated plainly here so the record is accurate: this was a dead end, not the resolution.

4.4 What finally cracked it

Two changes made the actual root cause visible:

- Commit **d2e281f** added server-side logging of *only the pathname* of the generated presigned URL (new URL(`presignedUrl`).`pathname`) — deliberately never the query string, since that carries the SigV4 signature. This is what disproved the doubled-bucket theory: the logged pathname was always correctly formed.
- Reading Vercel's runtime logs directly (rather than only reasoning about the code) surfaced the `TypeError [ERR_INVALID_CHAR]: Invalid character in header content ["authorization"]` stack trace on the image-upload path. That stack trace was the concrete evidence connecting the 500 error to a malformed credential string, which in turn explained why the presigned-URL (video) path was failing for the same underlying reason but through a completely different failure mode.

5. The Fix

Commit **06b2db2** applies `.trim()` to all five R2 environment variables (`R2_ENDPOINT`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_BUCKET_NAME`, `R2_PUBLIC_URL`) at the point they are read in `src/lib/media/r2.server.ts`, before they are used to construct the S3 client, the bucket name, or the public URL. This makes any future whitespace/newline copy-paste artifact in the Vercel dashboard harmless regardless of what is actually stored there, without needing to touch or re-enter the Vercel environment variables themselves.

Supporting Commits (verified against ``git log``)

52e071c	chore(r2): add CORS configuration helper and script
11d9736	fix(r2): disable auto CRC32 checksum on presigned PUT uploads
4c086ac	fix(r2): normalize R2_ENDPOINT that includes bucket name as path suffix
d2e281f	diag(r2): log presigned URL pathname to surface doubled-bucket paths
06b2db2	fix(r2): trim R2 env vars to guard against ERR_INVALID_CHAR recurrence

6. Verification Evidence

- A hero video upload was performed end-to-end after the fix; the resulting live URL returned HTTP **200** with `content-length: 6470592`. *(Reported by the engineer who performed the deploy; not independently re-verified while writing this report.)*
- `tsc --noEmit` and `eslint` were reported clean after the fix. *(Re-run and confirmed clean as part of preparing this report — see Testing.)*
- Production logs were reported to show no further 4xx/5xx responses on the affected routes after the fix deployed. *(Unverified in this report — Vercel runtime logs were not independently pulled while writing this document.)*

A note on the Cloudflare R2 dashboard “Objects / Size” columns.

The R2 Overview page's object count and total size are driven by a billing/usage report that can lag by hours and may show 0 B even right after a successful upload. This is a reporting-pipeline delay, not evidence of a failed write. The bucket's live object list (not the Overview summary) is the authoritative view of what has actually been stored.

7. Lessons & Preventive Measures

- **Always `.trim()` credential values read from environment variables.** Dashboards and clipboard managers can silently introduce leading/trailing whitespace that is invisible in a UI text field but breaks byte-sensitive operations like header construction and HMAC signing.

- **A “CORS error” in the browser can be a masked authentication failure.** Error responses frequently omit CORS headers by design (many servers only add them to successful/expected responses), so the browser's generic CORS message can hide a completely unrelated server-side rejection. Rule out an auth/signature failure before spending time on CORS configuration.
- **Browser-to-R2 direct PUT requests never appear in Vercel logs** — only the presigning request does. When a direct-to-storage upload fails, reproduce the signing logic locally with dummy credentials to inspect the generated request, rather than searching application logs for a request that was never routed through the application server.
- **Do not paste Cloudflare's “S3 API” endpoint URL (which includes the bucket name) directly into an SDK endpoint configuration value** when the SDK is also configured with `forcePathStyle`, which appends the bucket itself.
- **Sensitive Vercel environment variables cannot be read back via `vercel env pull`.** Since credential values can't be inspected after the fact, safe, credential-free diagnostic logging (pathnames, error types, host-only warnings) needs to be designed in up front, not added reactively once something breaks.

Part 2 — Subscription, Quota & Cost Review

This section reflects what could be directly observed via the Vercel CLI and the project's own source/configuration at the time this report was written. Anything that could not be verified through available tooling is explicitly labeled **unverified** rather than assumed.

Vercel

Plan tier: *unverified*. The Vercel CLI (`vercel teams ls`, `vercel project ls`) confirms the account/team "Omoruyi Idehen's projects" and lists projects, but does not expose the team's billing plan (Hobby vs. Pro) without deeper API access. The plan tier should be confirmed directly in the Vercel dashboard's billing settings.

Projects on the account: the CLI lists **5 projects** currently under this team: `jayshop-retail-platform`, `rent-hub-1bac`, `my-portfolio`, `driving-school-app-aaaj`, and `driving-school-app-xqpi`. The claim that 3 projects are queued for deletion could not be confirmed via CLI tooling in this session and should be treated as **unverified** — please confirm directly in the Vercel dashboard's project settings.

Serverless function limits & why the upload architecture matters

This app's admin hero **image** uploads are still routed through the Next.js server: the browser posts multipart form data to an API route, which buffers it and calls `saveUploadedFile` → `saveToR2`. That path consumes serverless function execution time for the duration of the upload and is subject to Vercel's request body size limit for serverless/edge functions. Hero **video** uploads deliberately avoid this: the browser requests a presigned URL from a lightweight API route, then uploads the video bytes directly to R2, bypassing the Next.js server (and its body-size and execution-time constraints) entirely. This is precisely why the presigned-URL architecture exists for video and not just for images — video files are far more likely to approach or exceed a serverless body-size limit than typical product/hero images.

- **Function execution time / payload size:** exact current limits depend on the confirmed plan tier (Hobby vs. Pro have different execution duration and function memory ceilings, and request body size limits have changed across Vercel platform versions). Because the plan tier itself is unverified here, the exact numeric ceiling is not stated as fact in this report — confirm current limits for the account's actual plan in the Vercel dashboard before relying on a specific number.
- **Practical risk:** as long as hero image uploads continue to flow through the Next.js server, unusually large images (or a slow client connection) could approach the function's body-size or execution-time ceiling. Video uploads are not at this risk because of the direct-to-R2 design.
- **Bandwidth allowance and build minutes:** unverified exact figures for this account's plan. On typical Vercel plans, exceeding included bandwidth or build-minute allowances results in either automatic metered overage billing (Pro/paid plans) or a hard block on further usage until the next billing cycle (Hobby plan) — confirm which applies once the plan tier is verified.

Compliance risk: commercial use on the Hobby plan.

Vercel's Hobby plan terms prohibit commercial use of hosted projects. Jays Shop is a retail / e-commerce platform that processes real transactions, which is a commercial use case. If the jayshop-retail-platform project is currently on the Hobby plan (unverified in this report), that is a real compliance risk that should be resolved by upgrading to a paid plan, independent of any technical quota concern.

Cloudflare R2

Current usage (as reported from the Cloudflare dashboard at the time of this incident, not independently re-queried while writing this report): approximately **71 Class A operations** and **30 Class B operations**, with total stored data effectively near zero plus the one hero video just uploaded (~6.2 MB). This usage is stated as observed/reported and should be treated accordingly.

How R2 pricing works, in practical terms:

- **Storage:** billed per GB-month of data actually stored in the bucket.
- **Class A operations** (writes: uploads, lists, and similar mutating/metadata calls) are priced per million operations and are the more expensive operation class.
- **Class B operations** (reads: e.g. serving/fetching an object) are priced per million operations and are considerably cheaper than Class A.
- **Egress (data transfer out) is \$0** on R2 regardless of volume. This is the standout difference versus most S3-compatible storage, and it is specifically why R2 is the right backend for serving video: a popular hero video watched thousands of times would rack up meaningful egress charges on a provider that bills for it, but costs nothing extra to serve here beyond the (cheap) Class B read operations.

Free tier thresholds (per Cloudflare R2's published free tier, at the time of writing): 10 GB-month of storage, 1 million Class A operations/month, and 10 million Class B operations/month, at no charge. Against that, current usage of 71 Class A and 30 Class B operations and near-zero storage is a tiny fraction of a percent of the free allowance — there is effectively no near-term risk of exceeding the R2 free tier at this traffic level.

Realistic cost estimate at low/medium traffic: for a single-store retail catalog with a modest number of hero images/videos and moderate storefront traffic, monthly storage is likely to stay in the low single-digit GBs, and monthly operations are likely to stay in the tens-of-thousands to low-hundreds-of-thousands range for both classes — comfortably inside the free tier. Even at meaningfully higher traffic (say, an order of magnitude more reads), R2's zero-egress model keeps the marginal cost of serving more video views close to the underlying Class B operation cost, which is inexpensive per operation. A firm dollar figure for "medium traffic" is not stated here since it depends on actual traffic volume, which was not measured as part of this report.

Neon / Postgres (DATABASE_URL)

Provider: code comments in `src/lib/prisma.ts` explicitly identify the production database provider as **Neon** (referenced by name, in the context of its idle-suspend behavior). No connection string, host, or credential is reproduced in this report.

- **Compute hours / auto-suspend:** Neon's free tier suspends compute after a period of inactivity. This is directly referenced in the codebase: `prisma.ts` raises `connect_timeout` specifically because “our DB provider (Neon) suspends its compute when idle and can take several seconds to resume on the first connection after a quiet spell,” which previously surfaced as spurious “Can't reach database server” errors on cold starts.
- **Connection limits:** the same file caps Prisma's per-instance connection pool at `connection_limit=5` specifically to avoid Vercel's serverless model (many concurrent function instances, each with its own pool) exhausting the database provider's total connection budget — a known constraint on free/shared-compute Postgres tiers.
- **Storage cap and branch limits:** unverified in this report. Neon's free tier has a fixed storage ceiling and a limited number of branches/projects; the account's current usage against those limits was not queried (this would require direct access to the Neon dashboard/API, which was out of scope for this review).

Postgres SSL-mode deprecation warning: the codebase was searched for any SSL-mode-related warning or deprecation notice on the realtime inventory listener (`src/lib/realtime/pgListener.ts`, which uses the standalone pg driver for LISTEN/NOTIFY). No such warning is present in the connection logic, error handling, or comments in that file as currently written. This is reported as **unverified / not found** rather than confirmed — if this warning was observed directly in Vercel runtime logs, it was not reproducible from the repository alone and should be re-checked against live logs.

Recommendations

Priority	Recommendation
Verify first	Confirm the actual Vercel plan tier (Hobby vs. Pro) directly in the dashboard. If Hobby, this is both a compliance issue (commercial-use restriction) and a quota-risk issue — treat as the top priority to resolve.
Upgrade if Hobby	If confirmed on Hobby, upgrade to a paid Vercel plan given Jays Shop's commercial/e-commerce nature, independent of current traffic volume.
Watch	Neon compute-hour usage and connection-pool pressure as traffic grows; the app already mitigates this with a capped connection pool and retry logic, but sustained higher traffic will need the actual Neon plan/usage confirmed.
Watch	Vercel serverless function body-size/time limits for the remaining server-mediated image-upload path, especially if larger hero images or bulk uploads are introduced.
Safe to leave alone	Cloudflare R2 usage and cost: current usage is a small fraction of the free tier, and R2's zero-egress model scales favorably for video-heavy growth without a redesign.
Safe to leave alone	The R2 credential-trimming fix and CORS configuration: both are in place, narrowly scoped, and verified against the code as written.

Rough monthly cost estimate for a “properly production” setup (order-of-magnitude, not a quote): a paid Vercel plan (Pro tier, per-seat plus usage) is typically in the tens of dollars per month at this project's likely scale; Cloudflare R2 at current or moderately higher usage is likely to remain at or near \$0/month given how far usage sits below the free tier and R2's zero-egress pricing; a paid Neon compute tier (to avoid cold-start suspension and raise storage/connection limits) is typically in the tens of dollars per month. Altogether, a modestly-provisioned production setup across all three services is plausibly in the neighborhood of low-to-mid double-digit dollars per month, with R2 being the smallest line item by a wide margin. This is an estimate for planning purposes only; it is not based on an actual quote or invoice from any of the three providers.

This report contains no credentials, API keys, account IDs, or connection strings. Bucket and service names are included where relevant for clarity.